

TUTORIAL SESSIONS



5210COMP:

Software Engineering for Games

Module Leader: Dr Chris Carter

Tutorial 06: SFML Tutorial #04

Input Control Systems

Important: Finish all tasks in the previous tutorial before starting these tasks

Tutorial Overview

This tutorial will help you to develop a InputDevice that obtains input from the mouse and keyboard. In this tutorial you will:

- The **First Task** create a new InputDevice that represents keyboard and mouse input via a virtual game pad.
- The **Second Task** is to add an instance of the KeyboardAndMouseInputDevice to the Game so it can be accessed from anywhere – we will be using the ServiceLocator Pattern for this.
- The **Third Task** Use the InputDevice in your gameplay logic to move objects around on screen!

Task 1: Create a Keyboard and Mouse Input Device

In order to get input into the SEG Engine, we must create an input device that derives from the **InputDevice** class, which is already part of the engine code.

The data structures approach to input handling is to use a `std::map` (a map data structure, which is an associative structure) to associate input events with actions (key-value pairs) so that we abstracted our control scheme; in this module, we will show an alternative OO approach to input handling.

This time around, instead of game-specific actions, **we are going to map our controls to a virtual game pad** (PS5 / XBOX Layout) so that our game is not dependent on a particular set of keystrokes or mouse movements.

We could also write future input device instances for other types of controllers such as the Xbox Pad or PS5 Pad.

Therefore, in this tutorial we are going to write a combined Keyboard and Mouse Device that will read the input from our Keyboard and Mice using SFML's built in libraries for Physical Input and then map these to a Virtual Gamepad.

The Gamepad State data is in the InputDevice class already, as are the Enumerations which map to the virtual buttons/triggers/bumpers/sticks/digital pads of our gamepad device.

Last time around, we also took an event driven approach to input handling – this time around, we will be using Polling – that is checking the state of the virtual pad each time, by checking the state of the hardware pads and then setting the virtual state to match the physical state.

For example, we could say that the Virtual D-Pad up is pressed up when the W key is pressed on our Keyboard, or the Right Analogue Stick is held right, when we move the mouse to the right.

Therefore, we are going to:

- Create a sub-class of the Engine's Input Device class.
- Complete the class by writing the code which maps the physical state of our keyboard and mouse to the virtual game pad.
- We must then create an instance of the class in our Game class and pass a pointer to the InputManager so that the Input Manager is aware of the Class – this will automatically then start Polling the Pad (using a function called `ProcessInputs`)
- In our gameplay code, we will then retrieve this virtual game pad from the Input Manager using our System Context Provider system
- Use the functions of the InputDevice class to access and check the state of our virtual game pad and handle gameplay events such as movement, weapon fire etc using this device.

Let's start with the Inheritance relationship.

Step 1: Create a new class that inherits from InputDevice

Create a new class that inherits from `InputDevice` and call it **KeyboardAndMouseInputDevice**. You should place the class files in the **Game->Input** Folder.

Step 2: Add the required function declarations and definitions to complete the inheritance.

Add the required **ProcessInputs()** Function – use the concepts of inheritance covered in the lectures to help you generate the correct override for this function. By inheriting from this class you will also gain some data, which we will be using later.

Step 3: Add an KeyboardAndMouseInputDevice member to Game.cpp

Open **Game.cpp** and locate the code where the input manager is initialised.

Create a dynamically allocated temporary variable instance of the `KeyboardAndMouseInputDevice` using our **new** keyword and pointers (e.g. `new KeyboardAndMouseInputDevice()` assigned to a temporary variable)

Step 4: Set the InputDevice

Once you have an instance of an input device, you must set the `InputDevice` on the Input manager. Use the **SetInputDevice** function on the Game class' Input manager member variable (beginning with `m_`) to do this, passing your pointer to the `KeyboardAndMouseInputDevice` as the parameter.

Step 4: Fill in the ProcessInputs function

You have now setup your new class to be the input device for the game. However, we are yet to get any input.

The `ProcessInputs` function in your newly created class is where you must get input for the input device.

To help with this, there are two members of `InputDevice`. Both are Arrays.

- The First Array corresponds to the digital inputs (on or off) of our Virtual Device – this is an
- The Second Array corresponds to analogue inputs (-1.0 to 1.0 or 0.0 to 1.0).

Each Array holds an instance of either a `DigitalInput` object or an `AnalogueInput` object – these objects have functions that allow us to access and modify (read and write) the state of the virtual game pad.

Our task is to come up with an input control scheme for our Game which maps these Array elements to a Physical Keyboard key or Mouse Action.

The analogue inputs array can be accessed using the **E_AnalogueInput** enum, which has the corresponding enum values.

- E_AnalogueInput_LeftTrigger,
- E_AnalogueInput_RightTrigger,
- E_AnalogueInput_ThumbLX,
- E_AnalogueInput_ThumbLY,
- E_AnalogueInput_ThumbRX,
- E_AnalogueInput_ThumbRY

The digital inputs array can be accessed via the **E_DigitalInput** enum:

- E_DigitalInput_DpadUp,
- E_DigitalInput_DpadDown,
- E_DigitalInput_DpadLeft,
- E_DigitalInput_DpadRight,
- E_DigitalInput_Start,
- E_DigitalInput_Back,
- E_DigitalInput_LeftThumb,
- E_DigitalInput_RightThumb,
- E_DigitalInput_LeftShoulder,
- E_DigitalInput_RightShoulder,
- E_DigitalInput_A,
- E_DigitalInput_B,
- E_DigitalInput_X,
- E_DigitalInput_Y,

The reason for these enumerations is to give us a friendly name for our array elements (e.g. rather than accessing the array via element 0, we would instead access it via E_DigitalInput_DpadUp (e.g. this->m_DigitalInputs[E_DigitalInput_DpadUp] – that way we know this element will store the D-Pad's Up button just from reading our source code.

We will use C++ 03 Enum's rather than new Scoped Enums as the old Enum values can be directly used with an array.

See here for a discussion of this concept:

<https://stackoverflow.com/questions/404231/using-an-enum-as-an-array-index>

These buttons represent all the buttons you would typically find on a modern controller.

Therefore, in the ProcessInputs function:

- Come up with a mapping for the four movements of the D-Pad and some Analogue Buttons.

- Try and use the **sf::Keyboard** and **sf::Mouse** classes from SFML to poll the state of the keyboard and mouse and then associate it with the virtual game pad by setting the state of the relevant object in one of the two arrays – use the enum values to access the array.

In the case of Digital Input, the SetPressed function takes a Boolean value, so we could set this to true or false. sf::Keyboard has functions which given a physical keyboard key will return true or false whether it is currently pressed – **you need to wire these two concepts together.**

If you do this right, to map the D-Pad actions to the keys WSAD for example, this would be just four lines of code in your process input function.

- NOTE: Mouse Actions will be more involved than Keyboard Actions!
- The following SFML documentation and discussion on Polling/Events will be useful:
 - <https://www.sfml-dev.org/tutorials/2.5/window-inputs.php>
 - <https://gamedev.net/forums/topic/661781-keyboard-input-poll-push-or-pull/5186732/>
 - <https://gameprogrammingpatterns.com/state.html>
 - https://www.sfml-dev.org/documentation/2.5.1/classsf_1_1Keyboard.php
 - https://www.sfml-dev.org/documentation/2.5.1/classsf_1_1Mouse.php

Task 2: Working with the Input Manager class

To use the input in our game we must grab the InputManager from the Engine.

Let's add input to our Animated Sprite Game Object. We will introduce a really important feature of working with game engines, which is accessing objects that we haven't passed by parameter into our function.

Things like Input, Time, Windows etc. are used by many, many classes – rather than passing them around as function parameters (which would mean they would be in most functions), or putting them in as member variables of a class (e.g. via Aggregation) we use other methods to do so.

In this case, we will introduce our First Software Design Pattern – **The Service Locator pattern**.

It is suggested that you read through the following article before attempting this task, so you understand what we are trying to implement.

<https://gameprogrammingpatterns.com/service-locator.html>

Step 1: Obtain a pointer to the InputManager

We will be using the **GameObjectWithAnimatedSprite** Game Object you created earlier in the module as a test for input (you can then comment this code out later).

The SEG Engine has a class that will help us get the pointer to important manager classes.

It is called C_SysContext.

Include the following files:

```
"../Engine/System/Context/SysContextProvider.h"  
"../Engine/Input/InputManager.h"  
"../Engine/Input/InputDevice.h"
```

In order to get a pointer to the Input Manager you must call the following function:

```
InputManager* pInputManager = C_SysContext::Get<InputManager>();
```

And then from the input manager, get a pointer to the current InputDevice:

```
InputDevice* pInputDevice = pInputManager->GetInputDevice();
```

You can either do this once per frame in the Update Function, or grab the InputDevice and store it as a class member of the Class so it can be used across all functions – the first option has a slight performance cost, but it won't be enough to affect your game's frame rate.

Step 2: Getting input values.

The InputDevice system has the arrays of objects. Each Object has functions to check the state of the digital buttons on that device, as well as functions belonging to Input Device itself:

OnPressed() – Will be true for the first frame a button is pressed only.

IsDown() – will be true as long as the button is pressed.

IsUp() – will be true as long as the button is not pressed.

You must use the two Enumerations you saw earlier to access the individual buttons.

The function `GetButtonInput` will return the button object (have a look in the `InputDevice.h` file to see how this function works with the array of digital objects) , so we can directly call its function by accessing these classes using the class member access syntax (the full stop or `.`).

e.g.:

`pInputDevice->GetButtonInput(E_DigitalInput_DpadRight).OnPressed()`

HINT: All three functions return Booleans, so can be used with conditional statements such as *if* and *if-else*.

In the case of Analogue input, rather than true-false statements, we get an actual value.

There is a single function to access the analogue input values and that is:

GetValue() – will return a float between -1.0 and 1.0 or 0.0 and 1.0 depending on the Input button.

As above, you must use the *enum* introduced earlier to access the individual inputs:
For example:

`float x_pos = pInputDevice-> GetAnalogueInput (E_AnalogueInput_ThumbLX).GetValue();`

HINT: once you have the floating-point value `x_pos`, you can use this with conditional statements by testing the value with `>`, `<`, `>=`, `<=`, `==` and `!=`

Task 3: Using Input to move game objects

This task is simply, try to use input to move your animated sprite around the screen! – think about your x and y locations and feed them into the move function of the animated sprite from Tutorial 05 (or one of the `GameObjectWithSprite` if you haven't got an animated sprite).

It is suggested that you use `sf::Vector2f` object to help you do this, rather than individual x and y components

e.g.

```
sf::Vector2f tup = sf::Vector2f(0.0f,-1.0f);
```

creates a movement which goes in the Positive Y axis.

Working with whole vectors rather than components of a vector makes the math much easier.

There are two locations you can add this – it all depends on how many objects you want to move around – it can go either in a game state, or in the game object (we could also use inheritance to create a further sub class? e.g., moving animated sprite?)

You can put this code in either the update function of the class you want to move (e.g. the animated sprite) or if you want to move a specific instance of an object, put the code in `GameStateTest` instead – we will be commenting this code out, we are just using it as a **Unit Test** for our Input Device functions.

If this is not making sense, ask the module leader to help explain this to your development group.